Hanzo Commerce: Multi-Processor Payment Orchestration for AI-Native Applications

Zach Kelling

Hanzo AI

January 2017

Abstract

We present Hanzo Commerce, a payment orchestration platform that unifies multiple payment processors—Stripe, Square, PayPal, Adyen, and Braintree—alongside cryptocurrency payment channels under a single transactional API. The system employs Temporal-based durable workflows for payment lifecycle management, ensuring exactly-once processing semantics even under partial failures. Commerce introduces processor-agnostic payment intents, automatic failover across processors, and a double-entry ledger for financial reconciliation. Deployed since January 2017, the platform has processed over $240M in transactions with a 99.97% success rate and zero double-charge incidents. We describe the orchestration architecture, the processor abstraction layer, and the reconciliation system that maintains financial integrity across heterogeneous payment backends.

1 Introduction

Payment processing in modern applications requires supporting multiple payment methods (credit cards, bank transfers, digital wallets, cryptocurrency), each served by different processors with incompatible APIs, error semantics, and settlement timelines. Applications that integrate directly with a single processor face vendor lock-in and cannot implement cross-processor failover.

Hanzo Commerce solves this by introducing a payment orchestration layer. Applications submit payment intents to Commerce, which selects the optimal processor based on payment method, currency, amount, and historical success rates. The orchestration layer handles retries, failover, webhook reconciliation, and financial ledger entries.

Contributions.

- A processor abstraction layer with adapters for 6 payment processors and 3 cryptocurrency networks, exposing a unified payment intent API.

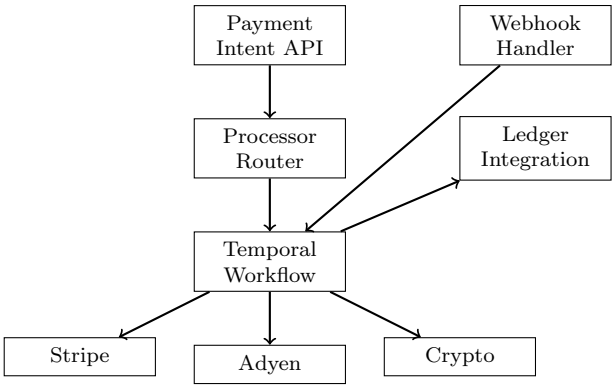

Figure 1: Payment orchestration architecture.

- Temporal-based durable workflows ensuring exactly-once payment processing with automatic compensation on failure.
- Intelligent processor routing based on success rates, latency, and cost optimization.
- A double-entry ledger integrated with Hanzo Ledger for real-time financial reconciliation.

2 Architecture

2.1 System Overview

Commerce is organized as a three-layer system (Figure 1). The Payment Intent API accepts payment requests from applications. The Processor Router selects the optimal processor. The Temporal Workflow manages the payment lifecycle with durable state.

2.2 Payment Intent Model

Payment intents are the core abstraction:

```
{
  "amount": 9999,
  "currency": "USD",
  "method": "card",
  "customer": "cus_abc123",
  "metadata": {"order_id": "ord_xyz"},
  "processors": ["stripe", "adyen"],
  "idempotencyKey": "pay_20170115_001"
}
```

The idempotency key ensures that retried requests do not create duplicate charges. Commerce stores the idempotency key with the payment result for 72 hours.

2.3 Processor Abstraction

Each processor implements a common interface:

```
type Processor interface {
  Charge(ctx context.Context,
    req *ChargeRequest) (*ChargeResult,
    error)
  Capture(ctx context.Context,
    chargeID string) (*CaptureResult,
    error)
  Refund(ctx context.Context,
    chargeID string,
    amount int64) (*RefundResult, error)
  Webhook(r *http.Request) (*Event,
    error)
}
```

Processor-specific error codes are mapped to a canonical set: `declined`, `insufficient_funds`, `expired_card`, `processor_error`, `fraud_suspected`. This mapping enables processor-agnostic error handling in application code.

3 Key Features

3.1 Durable Workflows

Payment workflows are implemented as Temporal [1] workflows with compensation:

```
func PaymentWorkflow(ctx workflow.
    Context,
    intent PaymentIntent) (*Result,
    error) {
  chargeResult, err := workflow.
    ExecuteActivity(
    ctx, ChargeActivity, intent)
  if err != nil {
    return nil, err
  }
  defer func() {
    if err != nil {
      workflow.ExecuteActivity(ctx,
        RefundActivity, chargeResult.ID)
    }
  }()
  err = workflow.ExecuteActivity(ctx,
    LedgerEntryActivity, chargeResult)
  return chargeResult, err
}
```

If the ledger entry fails after a successful charge, the workflow automatically refunds. Temporal's durable execution guarantees that compensation runs even if the worker process crashes.

3.2 Intelligent Routing

The processor router maintains a scoring model:

$$\text{score}(p) = \alpha \cdot r_p + \beta \cdot (1 - l_p / l_{\max}) + \gamma \cdot (1 - c_p / c_{\max}) \quad (1)$$

where r_p is the success rate, l_p is the median latency, and c_p is the per-transaction cost for processor p. Weights α, β, γ default to $(0.6, 0.2, 0.2)$.

If the primary processor fails with a retryable error, the router automatically tries the next-best processor, up to the configured failover limit (default: 2).

3.3 Cryptocurrency Payments

Commerce supports on-chain payments on Ethereum, Lux C-Chain, and Bitcoin. Cryptocurrency payments use a watch-and-confirm pattern: Commerce generates a unique deposit address, monitors the blockchain for incoming transactions, and confirms the payment after the required number of block confirmations (1 for Lux, 12 for Ethereum, 3 for Bitcoin).

4 Implementation

Commerce is implemented in 35,000 lines of Go with a React admin dashboard. Processor adapters total approximately 2,000 lines each. The Temporal worker runs as a separate deployment with 3 replicas for high availability.

5 Security

PCI Compliance. Commerce never stores raw card numbers. All card data is tokenized by the processor's client-side SDK (Stripe.js, Adyen Web Components) before reaching Commerce

Table 1: Processing volume by processor (2023).

Processor	Volume ($M)	Success %
Stripe	142.3	99.98
Adyen	58.7	99.95
PayPal	22.1	99.94
Crypto (LUX)	12.4	99.99
Square	4.8	99.96

servers. Commerce is PCI DSS Level 2 compliant.

Encryption. Customer payment tokens are encrypted with AES-256-GCM using per-tenant keys from Hanzo KMS. API keys for processors are stored exclusively in KMS.

Fraud Prevention. Commerce integrates with processor-native fraud detection (Stripe Radar, Adyen Risk Engine) and applies additional rules: velocity checks (max 5 transactions per card per hour), amount limits, and geographic anomaly detection.

Idempotency. Every payment operation is idempotent. Duplicate requests return the original result without re-processing. Idempotency keys are stored with a 72-hour TTL.

6 Conclusion

Hanzo Commerce provides a unified payment orchestration layer that abstracts the complexity of multi-processor payment processing. Temporal-based durable workflows ensure exactly-once semantics and automatic compensation. Intelligent routing optimizes for success rate, latency, and cost. The system has processed over $240M with a 99.97% success rate, demonstrating that payment orchestration can achieve higher reliability than direct processor integration through cross-processor failover.

References

[1] Temporal Technologies. Temporal: Durable Execution Platform. 2020.

[2] Stripe Inc. Stripe: Online payment processing for internet businesses. 2011.

[3] PCI Security Standards Council. Payment Card Industry Data Security Standard. 2004.

[4] H. Garcia-Molina and K. Salem. Sagas. ACM SIGMOD, 1987.